

Resident State as an Executable System Representation

A bounded result for intervention, control, and scenario analysis

AUTHOR
RONNIE MACK

BUILD
V3 + SV1 / 2026

Abstract

State-Space Programming Methodology (SSPM) studies whether bounded dynamical programs can be represented as transformations over **resident matrix state** rather than rebuilt as objects, events, or solver-specific structures for each intervention. Its representation separates reusable affine structure from the nonlinear and order-sensitive behavior required to preserve the original transition:

$$L_t = S_t A^\top + U_t B^\top + b, \quad S_{t+1} = P_M(L_t + H_{ordered}(S_t, U_t, L_t, G_t)).$$

The completed V3 and Serving V1 cycles support a bounded claim: resident state makes large counterfactual batches and target edits direct when the alternative reconstructs object or event models. They reject a stronger claim that matrix representation alone creates the fastest numerical kernel. Manual vectorization and fused execution remain faster for residual-heavy transitions.

1. The claim

Let $S_t \in \mathbb{R}^{N \times d}$ contain typed state, with rows representing entities or independent scenarios, and let U_t contain controls or exogenous inputs. SSPM makes state and transition operators first-class executable objects. A , B , and b expose reusable affine dependencies; $H_{ordered}$ preserves nonlinear expressions, branches, updated-state reads, masks, and graph reductions; P_M applies declared projection and mode logic.

An intervention modifies a target state, control, capacity, constraint, or operator while the shared system representation remains resident. Candidate alternatives can be stacked as rows and propagated together. The intended benefit is reduced reconstruction and a common surface for rollout, composition, intervention, linearization, controllability, reachability, and model-predictive control.

This is not universal linearization. Exact finite-dimensional linear representations of general nonlinear dynamics are uncommon [1]. An unrestricted residual can always absorb the transition, but then the decomposition offers no leverage. A useful result requires semantic preservation, enough reusable structure, and lower manipulation cost before residual burden dominates.

2. Completed experiment program

The frozen V3 workload ladder contains an exact double integrator, a piecewise-affine bounded queue and scheduler, and a controlled Kuramoto graph with nonlinear coupling in the residual. Traditional object, scalar, manual NumPy, fused Numba, equivalent event, and SSPM forms receive identical states, controls, traces, horizons, precision, and materialization boundaries.

The restricted source compiler extracts affine terms and explicitly rejects hidden mutation, I/O, ambiguous aliasing, loops, and unsupported effects. Differential tests cover one-step and trajectory behavior. Mutation tests alter coefficients, dependencies, ordering, thresholds, projections, and modes. Control and reachability claims are workload-specific: exact for affine LTI, conservative for the queue, and local or empirical for Kuramoto.

A separate Serving V1 cycle models prefill/decode backlog, KV occupancy, admission, completion, drops, and accumulated latency work in a source-informed fluid scheduler. It tests whether resident state helps evaluate large batches of capacity, workload, and initial-state interventions. It is not a calibrated inference-engine simulator.

3. Results

Semantic gates passed across the frozen V3 corpus. Affine coverage was 100% for LTI, 55.2% for the queue, and 37.5% for Kuramoto. Generated NumPy, Numba, Torch CPU, and C++20 exact-affine paths agreed within maximum error 2.14×10^{-14} . At 2,048 scenarios and horizon 20, SSPM was 6.9x to 63.8x faster than object/scalar baselines and 13.9x faster than the equivalent queue event baseline. Resident intervention crossed reconstruction-heavy baselines at 256 scenarios for every LTI and queue horizon tested.

Control results preserved the same candidate objectives and selected policies, with no additional constraint violations. Exact LTI zonotopes and conservative queue intervals contained all tested trajectories. Kuramoto enclosures are explicitly nonformal.

The residual frontier is the decisive negative result. Across 50 configurations, resident SSPM did not beat the equivalent manual-vectorized update. At 2,048 scenarios and horizon 20, manual NumPy was about 8.3x faster for LTI and 7.6x faster for queue; fused Numba was about 68.0x and 44.6x faster. Representation alone did not create a superior kernel.

Serving V1 reproduced the same boundary across two process sessions and 5,280 retained timing samples. At 256 or more scenarios, object reconstruction divided by affine-residual replay was 36.17x at the median. Object reconstruction divided by resident intervention replay was 34.72x at the median. Manual vectorization won every measured serving cell.

4. Disposition

The evidence supports SSPM as a **resident-state representation and manipulation method**. It provides a direct route from reconstruction-heavy models to batched counterfactual execution and exposes analytical operators where the mathematics permits. It does not support universal speed, arbitrary-system linearization, or formal nonlinear reachability.

The next falsification target is explicit residual compilation and fusion. The open question is whether SSPM can preserve its inspectable semantic and analysis surface while approaching strong vectorized execution. Until that is shown, fused or direct vectorized paths should bypass generic affine-plus-residual replay when they are cheaper.

References

- [1] S. L. Brunton, B. W. Brunton, J. L. Proctor, and J. N. Kutz, "Koopman Invariant Subspaces and Finite Linear Representations of Nonlinear Dynamical Systems for Control," *PLOS ONE*, 11(2), e0150171, 2016. <https://doi.org/10.1371/journal.pone.0150171>
- [2] U. A. Acar et al., "An Experimental Analysis of Self-Adjusting Computation," *ACM TOPLAS*, 32(1), 2009. <https://doi.org/10.1145/1596527.1596530>
- [3] F. McSherry et al., "Differential Dataflow," *CIDR*, 2013. https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf

Evidence boundary. Local Apple Silicon experiments; bounded synthetic and source-informed workloads; no CUDA/Triton or production-serving claim. The public archive contains aggregate evidence, while raw traces and executable code remain private.